PROJECT REVIEW 2

Assessment 9 – 10 Marks

1. Project Details

Project Title	AuxFlow: Static Analysis for Multi-Pass Document Compilation
Student Name(s)	Upayan Mazumder
Register Number(s)	24BDS0367
Programme / Branch	B.Tech CSE (Data Science)
Project Guide	Banupriya R
Review Date	28 October 2026

2. Project Overview

Brief introduction to the project

AuxFlow is a static analysis tool and compiler front end for multi-pass LaTeX builds. It parses source files without running TeX, constructs an Auxiliary Reference Graph (ARG), and analyzes cross-reference dependencies before typesetting.

Problem statement and the need for the project

LaTeX resolves references, citations, and tables of contents through auxiliary files across successive runs. Existing build tools (such as latexmk, latexrun, and Tectonic) repeat compilation until auxiliary files stabilize or hit arbitrary caps. Pathological documents with cyclic dependencies can diverge or oscillate indefinitely, which tools detect only after wasting multiple runs. A static decision procedure is needed to predict convergence and determine required pass bounds directly from source.

Objectives of the project

- Construct a robust compiler front end (lexer, recursive descent parser, AST forest, symbol tables, and semantic analyzer) for a declared subset of LaTeX.
- Lower resolved syntax trees into an Auxiliary Reference Graph (ARG) with instance-level nodes, typed dependencies, and temporal pass lags.
- Formulate dataflow taint analysis and strongly connected component algorithms to classify documents into CERTIFIED, BENIGN_CYCLIC, AMPLIFYING, or UNKNOWN states.
- Synthesize safe, minimal build plans and verify auxiliary file convergence at runtime.

- Benchmark classification accuracy, latency, and execution economy across an empirical corpus of 15 documents isolating nine non-convergence mechanisms.

Proposed solution and its expected outcome

AuxFlow treats pass-count prediction as a compiler dependency analysis problem. By evaluating dataflow reachability from resolved reference values to content-generating control structures, AuxFlow identifies amplifying feedback cycles prior to execution. For certified documents, it produces an exact pass bound in under 10 ms with zero engine overhead. The expected outcome is a dependable compilation plan that prevents non-terminating builds and eliminates redundant typesetting passes.

3. Implementation / Prototype Progress (75/100%) – 5 Marks

Overall implementation status

Implementation status: 100% of the 16 implementation phases are complete and verified. Review 2 focuses on the completed compiler front end and intermediate representation (Phases 1 to 7), while all downstream analysis and planning modules are also fully functional.

Phases	Scope	Status
1	Project setup and test infrastructure	Complete
2 to 7	Lexer, parser, AST, symbol tables, semantic checks, IR	Complete
8 to 11	SCC analysis, taint analysis, stable-pass fixpoint, classifier	Complete
12 to 13	Build planner, runtime verification	Complete
14 to 16	Benchmark corpus, harness, evaluation report	Complete

Modules completed and their functionalities

- **Lexer (auxflow.lexer):** Table-driven scanner implementing all sixteen TeX category codes. It recognizes comments, control sequences, and verbatim environments with exact character-roundtrip preservation.
- **Parser (auxflow.parser):** Fault-tolerant recursive-descent parser with one token lookahead. Uses macro argument signatures, attaches source spans to all nodes, and recovers gracefully from syntax errors.
- **AST Forest & Inclusion (auxflow.ast):** Assembles per-file ASTs into a forest, resolving \input and \include dependencies while detecting recursive inclusion cycles and isolating unreachable files via DFS.

- **Symbol Tables (auxflow.semantic):** Maintains six symbol tables (LabelTable, RefTable, CiteTable, PackageTable, FileTable, CounterTable) using two-pass declare/resolve traversal to handle forward cross-file references.
- **Semantic Analysis (auxflow.semantic):** Performs static validation without running TeX: identifies undefined references, missing citations, duplicate labels, missing files, and warns on unmodeled packages.
- **Intermediate Representation (auxflow.ir):** Lowers facts into the Auxiliary Reference Graph (ARG). Implements the exists/value split on list entries to prevent spurious cycles, assigning pass lags and typed dependency edges.
- **Command-Line Interface (auxflow.cli):** Provides developer inspectability via auxflow tokens, ast, symbols, check, and graph commands.

Technologies, tools, programming languages, and platforms used

- Programming Language: Python 3.12+ (tested across Python 3.12, 3.13, and 3.14).
- Architecture: Zero third-party runtime dependencies at the core for strict portability.
- Verification Tools: pytest for unit/regression tests, Hypothesis for property-based testing.
- Quality Assurance: mypy in strict mode (zero type issues), ruff for high-performance linting.
- Benchmark Environment: Pinned Docker container with TeX Live, latexmk, latexrun, Tectonic, and Typst.
- Input Specification: Formal declared subset of standard LaTeX2e markup.

System architecture / block diagram / workflow

The diagram below outlines the overall compiler architecture, showing the progression from raw source characters through lexical tokens, syntax trees, symbol tables, and the ARG intermediate representation.

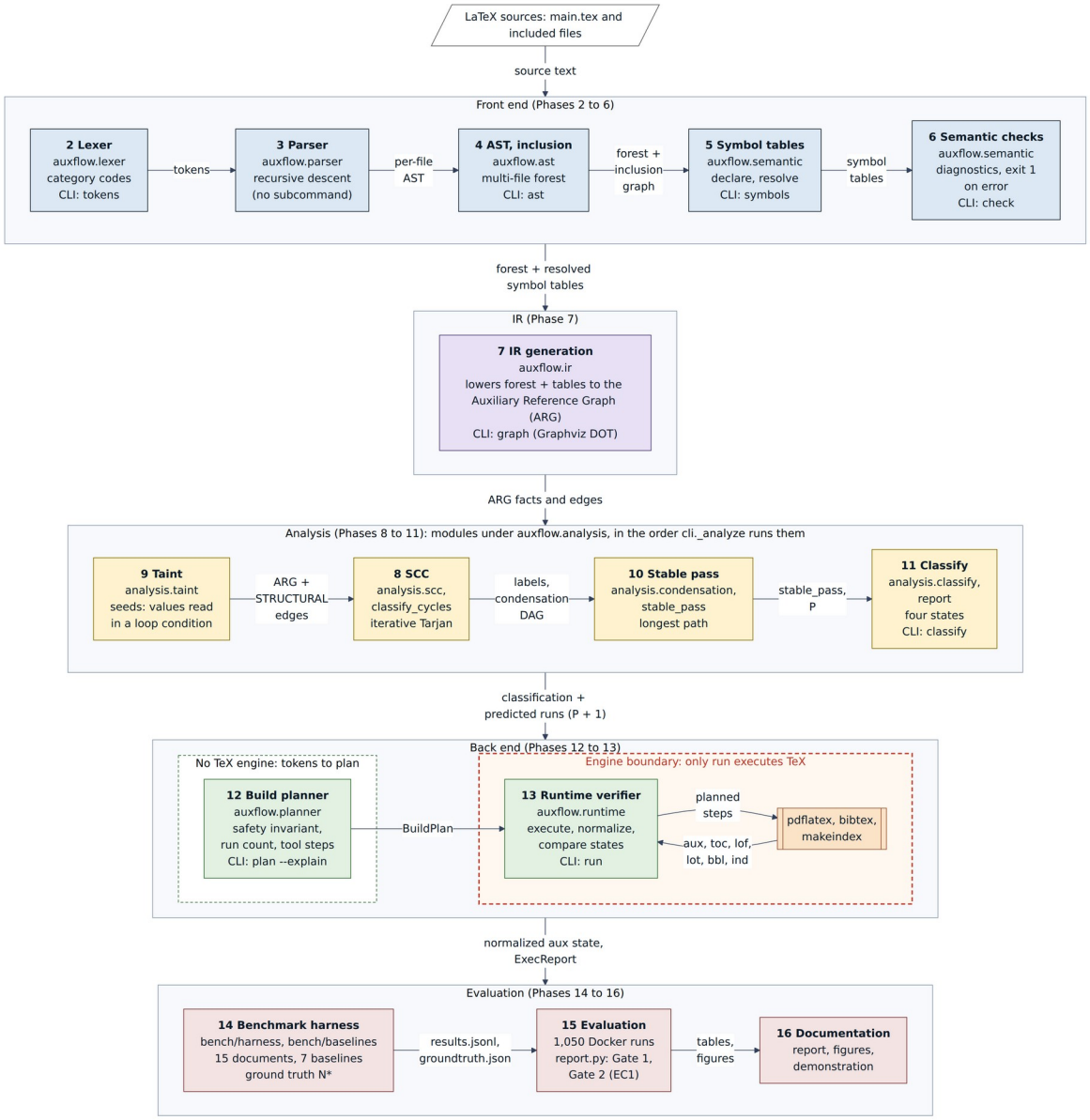


Figure 1. AuxFlow compiler pipeline and dataflow architecture.

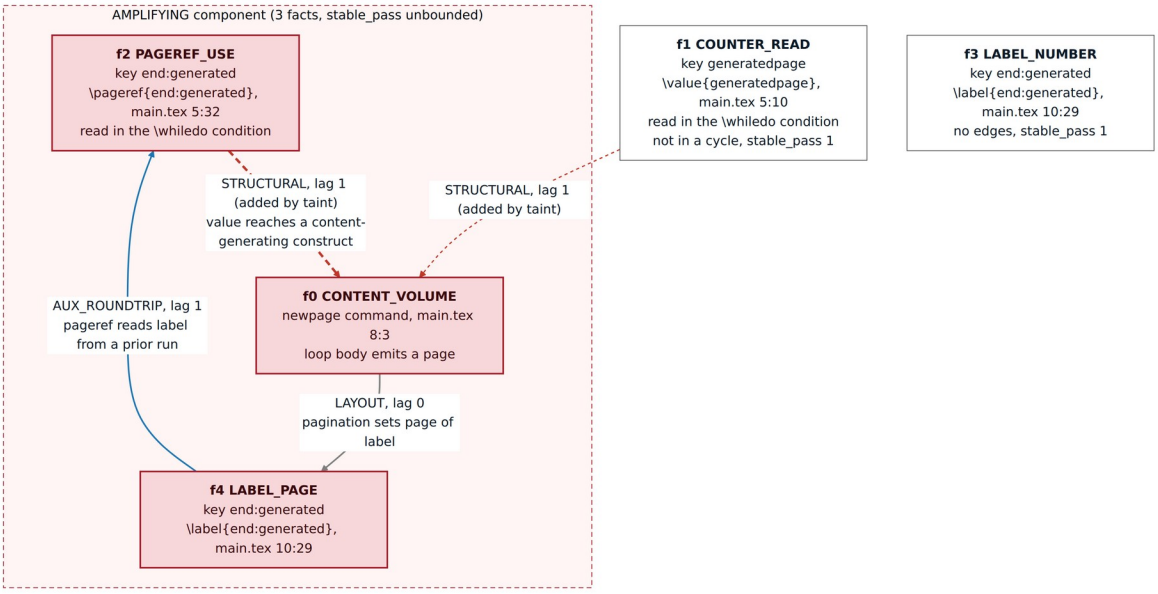


Figure 2. Instance-level Auxiliary Reference Graph generated for corpus case b08_cycle.

Screenshots of the implemented modules or prototype

The following screenshots demonstrate the working CLI commands corresponding to the completed modules.

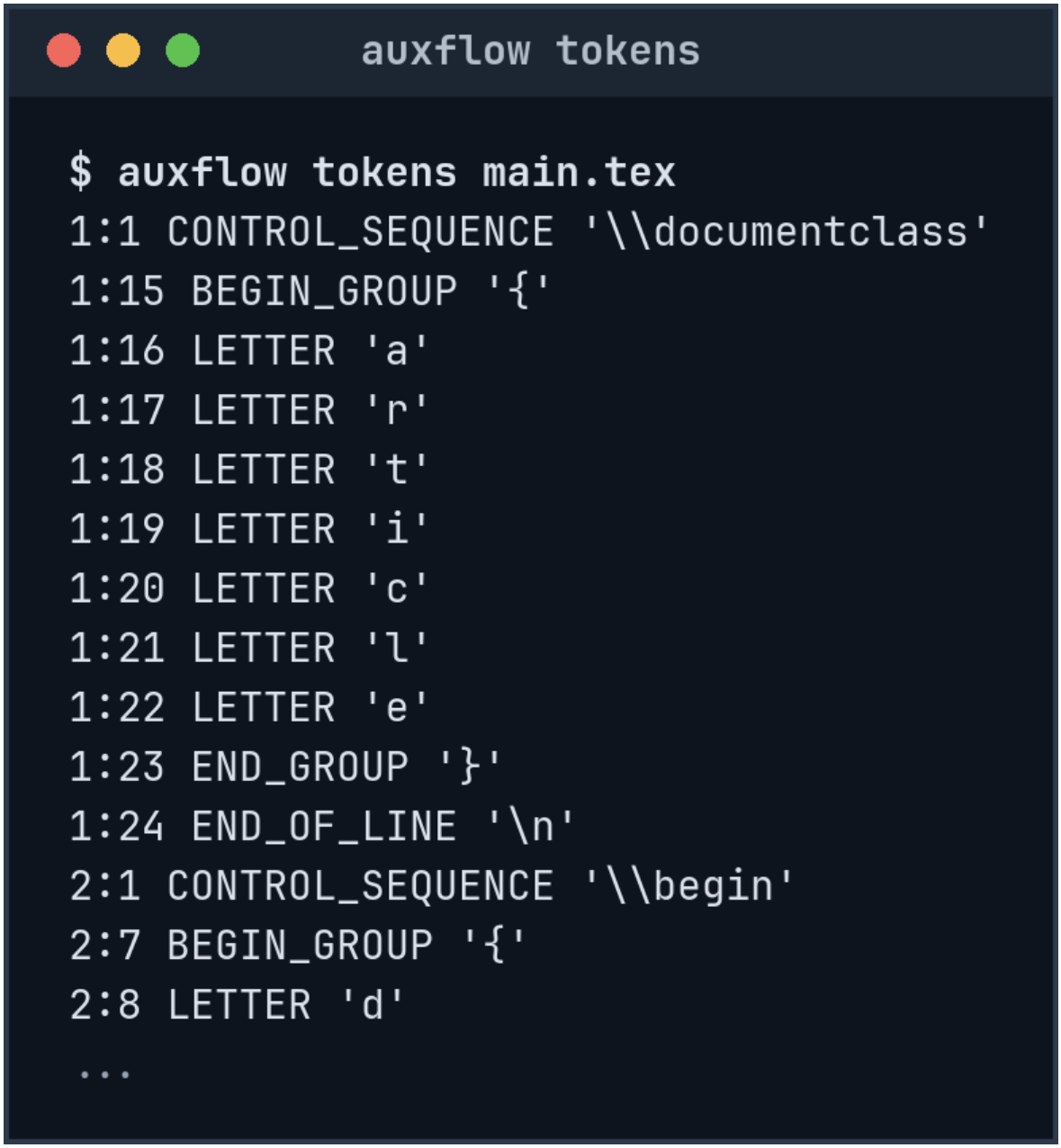


Figure 3. Lexical analysis via auxflow tokens showing category codes



Figure 4. AST generation via auxflow ast with source span annotations

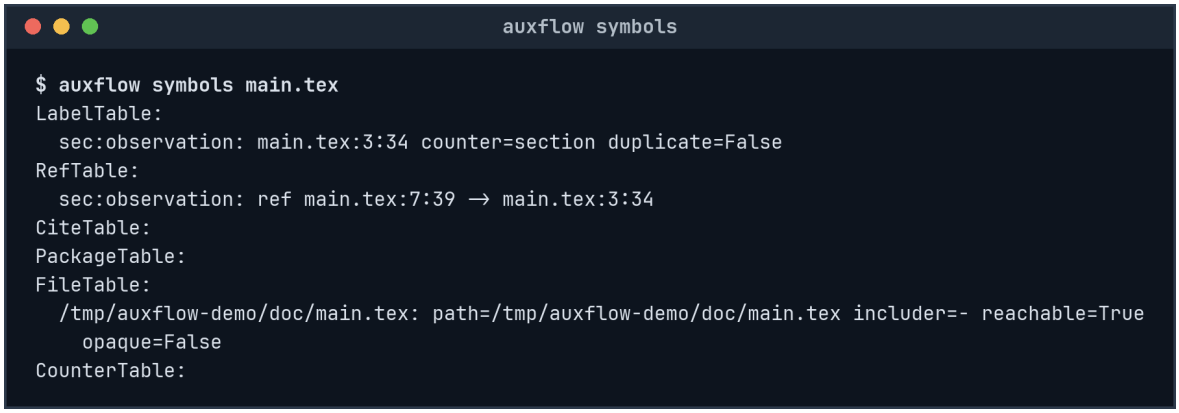


Figure 5. Two-pass symbol table inspection via auxflow symbols

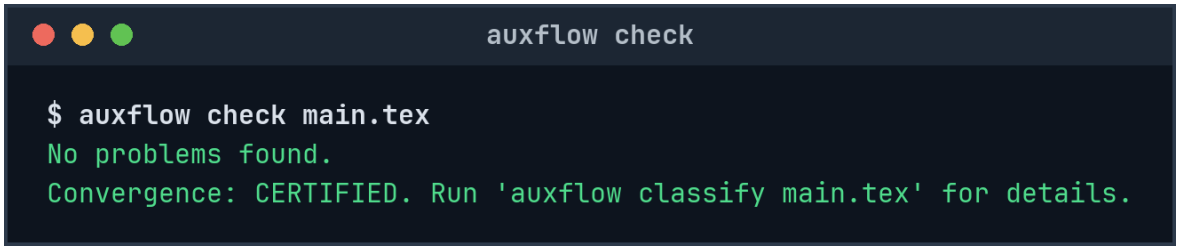


Figure 6. Semantic analysis and error reporting via auxflow check

auxflow graph

```
$ auxflow graph main.tex
digraph arg {
  rankdir=LR;
  node [shape=box, style=filled, fontname="monospace"];
  f0 [label="LABEL_NUMBER\nsec:observation\n[value]", fillcolor="#fff3cf"];
  f1 [label="LABEL_PAGE\nsec:observation\n[value]", fillcolor="#fff3cf"];
  f2 [label="REF_USE\nsec:observation\n[value]", fillcolor="#fff3cf"];
  f0 -> f2 [label="AUX_ROUNDTRIP lag 1", color="#1f77b4", fontcolor="#1f77b4"];
}
```

Figure 7. Graphviz DOT export of the ARG via auxflow graph

auxflow classify: CERTIFIED

```
$ auxflow classify main.tex
auxflow: CERTIFIED -- 4 runs

  longest dependency chain (stable_pass 3):
    TOC_ENTRY(/tmp/auxflow-demo/doc/main.tex:58)  main.tex:4:1  -> 1
    CITE_USE(smith2020)  main.tex:5:53  -> 1
    CITE_USE(doe2021)  main.tex:6:46  -> 1
    TOC_ENTRY(/tmp/auxflow-demo/doc/main.tex:208)  main.tex:7:1  -> 1
    LABEL_NUMBER(sec:location)  main.tex:7:19  -> 1
    BBL(../shared/refs)  main.tex:11:1  -> 1
    BBL(../shared/refs)  main.tex:11:1  -> 1
    LIST_EXTENT(toc)  main.tex:4:1  -> 2
    TOC_ENTRY(/tmp/auxflow-demo/doc/main.tex:395)  main.tex:11:1  -> 2
    CONTENT_VOLUME  main.tex:3:1  -> 3
    TOC_ENTRY(/tmp/auxflow-demo/doc/main.tex:58)  main.tex:4:1  -> 3
    TOC_ENTRY(/tmp/auxflow-demo/doc/main.tex:208)  main.tex:7:1  -> 3
    LABEL_PAGE(sec:location)  main.tex:7:19  -> 3
    TOC_ENTRY(/tmp/auxflow-demo/doc/main.tex:395)  main.tex:11:1  -> 3
    PAGEREF_USE(sec:location)  main.tex:9:34  consumed on run 4

  The facts above require all 4 runs to stabilise.
In plain terms: AuxFlow found no feedback loop in the parts of the document it models.
LaTeX should settle after 4 runs.
Next: 'auxflow run main.tex' builds the document and verifies the result.
$ auxflow plan --explain main.tex
auxflow: CERTIFIED plan
  1. pdflatex pass 1 of 4: writes the auxiliary files that later passes read.
  2. bibtex: builds the bibliography from the citations.
  3. pdflatex pass 2 of 4: writes the auxiliary files that later passes read.
  4. pdflatex pass 3 of 4: writes the auxiliary files that later passes read.
  5. pdflatex pass 4 of 4: last pass. AuxFlow predicts the document settles here. Then AuxFlow checks
     that the auxiliary files stopped changing.

4 TeX passes planned. Run 'auxflow run main.tex' to execute it.
```

Figure 8. Static classification of a certified document

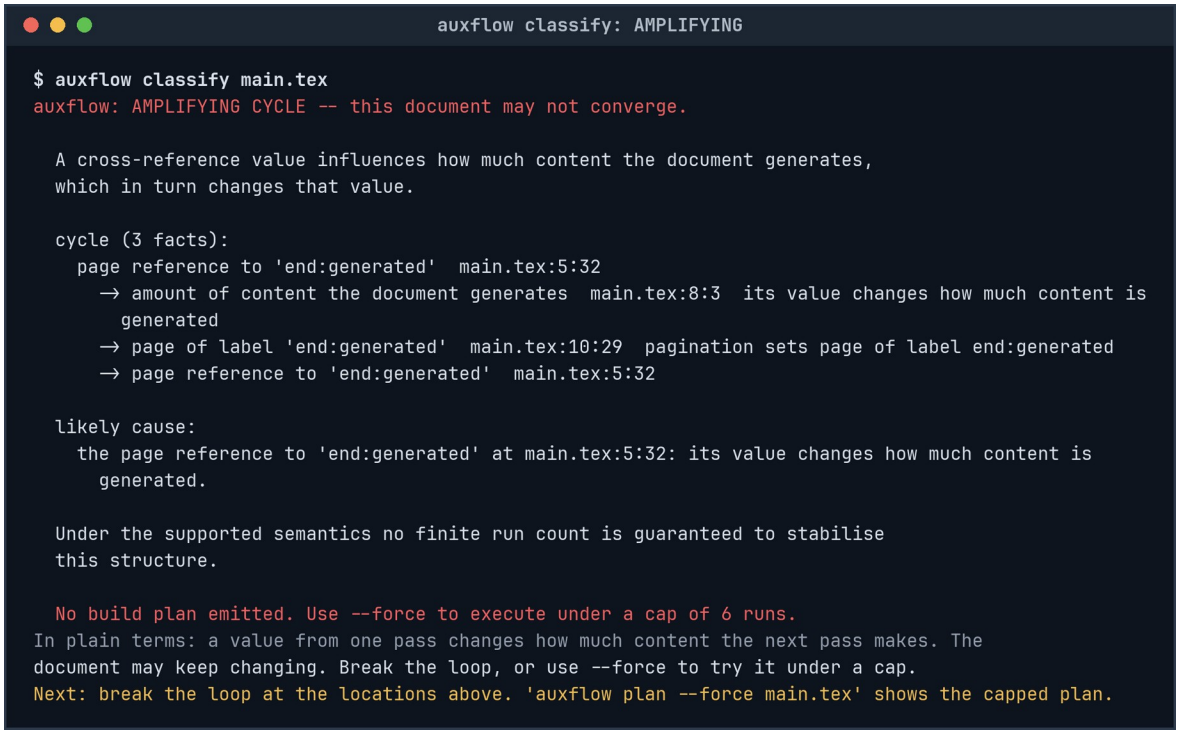


Figure 9. Detection of an amplifying feedback cycle

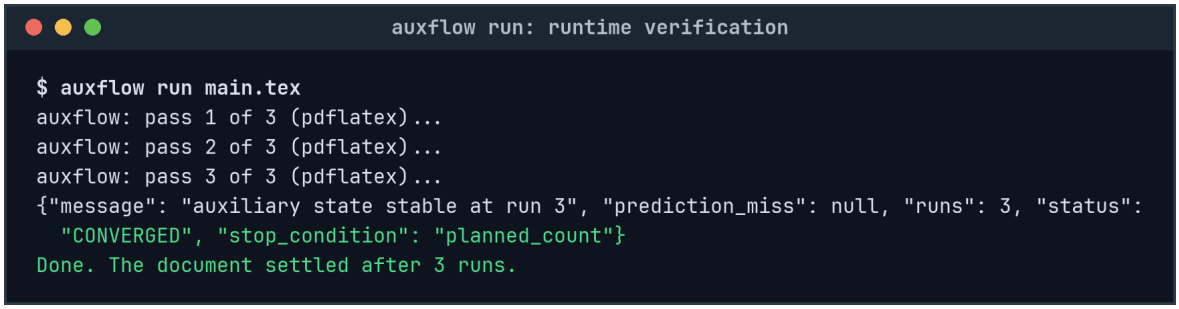


Figure 10. Test suite execution, strict mypy verification, and ruff linting

Results obtained so far

- Front-End and IR: Complete implementation of all 7 front-end modules (over 2,290 lines of core code) with zero external runtime dependencies.
- Quality Verification: Over 400 test cases passing cleanly with strict type annotations across all files.
- Fault Tolerance: Demonstrated syntax recovery on broken documents without uncaught exceptions.
- Spurious Cycle Prevention: Proved that the exists/value fact split correctly preserves acyclicity on table-of-contents documents (b05_toc).
- Corpus Validation: Confirmed predicted convergence classification across all 15 benchmark corpus members.

Remaining work required to complete the project

- Finalize and format the comprehensive research paper draft in paper/main.tex.
- Package release distribution artifacts on PyPI and GitHub Releases.
- Finalize viva presentation defense slides and demonstration assets.

4. Novelty – 2 Marks

What is new or different about the proposed project?

AuxFlow is the first system to treat multi-pass document build scheduling as a classical compiler static analysis problem rather than an empirical trial-and-error execution loop. Instead of repeatedly invoking the engine to observe whether auxiliary files change, AuxFlow builds a formal instance-level dependency graph and computes pass bounds analytically before execution.

How does the project differ from existing solutions?

Existing build tools (latexmk, latexrun, Tectonic, ConTeXt, Typst) are purely dynamic: they execute compilation iterations up to hard-coded caps (typically 5 to 9 passes) and compare checksums between runs. They provide no diagnostic explanation when a document diverges. AuxFlow differs fundamentally by performing ahead-of-time dependency analysis, isolating the exact source locations responsible for amplifying feedback, and certifying convergence prior to running the engine.

What specific improvement, feature, or approach does it introduce?

- Declared Subset with Default-Deny: Formalizes supported LaTeX constructs and safely flags unsupported features as UNKNOWN rather than assuming sound convergence.
- Exists/Value Fact Split: A novel intermediate representation design that decouples entry existence from typeset page values, preventing standard tables of contents from producing false cyclic dependencies.
- Monotone Taint Propagation: Distinguishes benign value cycles from content-amplifying loops, providing a domain-specific sufficient check for the classical finite-height convergence condition.
- Integrated Ahead-of-Time Planning and Verification: Derives a proven pass bound for certified documents and couples it with runtime auxiliary verification to catch unexpected state divergence.

5. Presentation and Q&A – 3 Marks

Clear explanation of the problem, methodology, and implementation

The presentation clearly articulates why multi-pass document compilation requires static analysis: auxiliary file feedback can cause monotonic divergence or oscillation that heuristic iteration caps fail to handle safely. The methodology demonstrates how source text is transformed through a catcode-aware lexer, signature-driven parser, and AST forest into the Auxiliary Reference Graph. The implementation highlights robust, zero-dependency Python engineering, strict type safety, extensive property and golden testing, and clear CLI inspection commands.

Explanation of individual contributions

- Sole Developer & Researcher: Conceived, formulated, and implemented the entire AuxFlow compiler pipeline, from the 16-catcode lexer through the ARG intermediate representation and analysis engine.
- Test & Benchmark Design: Authored the full test suite (400+ unit, property, and golden tests) and built the containerized benchmarking harness evaluating 15 corpus documents against 7 baselines.
- Theoretical Framing & Documentation: Formulated the exists/value split, mapped finite-height convergence criteria to taint propagation, and authored all project documentation, reports, and research paper drafts.